DevTrack

AI-Powered IT Project Management System — Complete Technical & Strategic Guide

Tejas Kumbhar · Avishkar Thombare · Akshay Ghule · Tejas Borhade · Krushna Sarode
Guided by: Dr. Shanti Guru · Mrs. Nilam Patil · Mrs. Sneha Kumbhar

Document Date: February 27, 2026 | Status: Pre-Development Planning

TABLE OF CONTENTS

1. What is DevTrack — Full Abstract & Vision	Section 1
2. System Architecture (End-to-End)	Section 2
3. How the AI Brain Works — Step by Step	Section 3
4. Jira Epics, Stories & Tasks (Full Breakdown)	Section 4
5. Tech Stack Recommendation	Section 5
6. Competitor Analysis — What Exists vs What You're Building	Section 6
7. Copyright & Novelty Assessment	Section 7
8. Honest Gaps & Risks	Section 8
9. Sprint-wise Development Roadmap	Section 9
10. Final Verdict	Section 10

📋 1. What is DevTrack — Full Abstract & Vision

DevTrack is an AI-powered project management system designed specifically for software development teams working in **Agile / Scrum** environments. Unlike traditional tools like Jira (which require humans to manually create and track everything), DevTrack *automates* the entire loop — from requirements to code to progress reports.

> **Core promise: Upload your SRS document → DevTrack reads it → Generates Epics, Stories, Tasks in Jira automatically → Monitors your GitHub code → Tells you exactly what is done and what isn't — all without manual updates.**

The 5 Core Capabilities

#	Capability	What It Does
1	📄 SRS Parser	Reads Software Requirement Specification documents and extracts structured requirements using AI
2	🧠 Story Generator	Converts raw requirements into Jira-ready Epics, User Stories, and Tasks with acceptance criteria
3	🔗 Jira Integration	Auto-creates and updates work items in Jira via REST API after Scrum Master approval
4	💻 GitHub Monitor	Watches your code repository, analyses commits and files to detect if a story is implemented
5	📊 Progress Dashboard	Real-time view showing module completion %, partial implementations, and overall project health

🏗️ 2. System Architecture (End-to-End)

```
┌──────────────────────────────────────────────────────────────────────────┐
DEVTRACK SYSTEM ARCHITECTURE │
└──────────────────────────────────────────────────────────────────────────┘
INPUT LAYER ┌──────────────┐ ┌──────────────┐ ┌──────────────┐ ┌──────────────┐ │ SRS / PDF
│ │ Budget & │ │ Deadline / │ │ Scrum Master│ │ Document │ │ Scope Docs │ │
Milestones │ │ Prompts │ └──────┬───────┘ └──────┬───────┘ └──────┬───────┘
└──────┬───────┘ └──────────────┴──────────────┴──────────────┴──────────────┘ │ ▼
PROCESSING LAYER — KNOWLEDGE BASE
┌──────────────────────────────────────────────────────────────────────────┐ │ Document
Parser (PDF/DOCX extraction) │ │ → Text Chunking → Vector Embeddings (OpenAI /
HuggingFace) │ │ → Vector Database (Pinecone / ChromaDB) │ │ → Semantic Search Index
│ └──────────────────────────────────────────────────────────────────────────┘ │ ▼ AI BRAIN
LAYER — LLM PIPELINE
┌──────────────────────────────────────────────────────────────────────────┐ │ RAG
(Retrieval Augmented Generation) │ │ Scrum Master Input → Retrieve context from KB │
│ → LLM Prompt (GPT-4 / Gemini) → Generate: │ │ • Epics (module level) │ │ • User
Stories (role-based, with acceptance criteria) │ │ • Tasks (technical sub-items) │ │
→ Output: Structured JSON / Jira-ready format │
└──────────────────────────────────────────────────────────────────────────┘ │
┌──────────┴──────────┐ ▼ ▼ REVIEW GATE AUTO-PUSH TO JIRA Scrum Master views Jira
REST API → Create generated items → Epic / Story / Task Approves / Edits → with auto-
labels & Confirms sprint assignment │ ▼ CODE ANALYSIS LAYER — GITHUB MONITOR
┌──────────────────────────────────────────────────────────────────────────┐ │ GitHub
Webhooks / Polling → Fetch changed files & commits │ │ AST Parser (Babel / Tree-
sitter) → Extract: │ │ • API routes defined │ │ • Components created │ │ • Functions
matching acceptance criteria keywords │ │ LLM Validator: "Does this code satisfy
```

```
Story X criteria?" │ │ → DONE / PARTIAL / NOT STARTED │
└──────────────────────────────────────────────────────────────────────┘ │ ▼ SYNC LAYER
Update Jira issue status → Log evidence (which file, which line) → Notify Scrum
Master of completions / blockers │ ▼ OUTPUT LAYER — DASHBOARD
┌──────────────────────────────────────────────────────────────────────┐ │ Real-time
React Dashboard showing: │ │ • Epic completion % with progress bars │ │ • Story
status: Done / Partial / Not Started │ │ • Which files implement which story
(traceability) │ │ • Sprint burndown (actual vs planned) │ │ • Risk alerts (blockers,
overdue stories) │ │ • Exportable PDF reports (like this document!) │
└──────────────────────────────────────────────────────────────────────┘
```

🧠 3. How the AI Brain Works — Step by Step

Step 1 — Document Ingestion

1	→	2	→	3	→	4	→	5
Upload		**Extract**		**Chunk**		**Embed**		**Store**
PDF/DOCX/TXT SRS file		Raw text from document		Split into 500-word segments		Convert to vectors		Save to vector DB

Step 2 — Story Generation (RAG Pipeline)

1	→	2	→	3	→	4	→	5
SM Input		**Retrieve**		**Prompt**		**Generate**		**Review**
Scrum Master types module name		Top-5 relevant chunks from KB		LLM prompt with context		Epics + Stories + Tasks JSON		SM approves → Push to Jira

Step 3 — Code Analysis (GitHub Monitor)

1	→	2	→	3	→	4	→	5
Webhook		**Fetch**		**Parse**		**Match**		**Update**
GitHub push triggers event		Changed files pulled		AST analysis of code		LLM: Does code = story criteria?		Jira + Dashboard synced

📌 4. Jira Epics, Stories & Tasks — Full Breakdown

Sprint labels: **S1** Sprint 1 **S2** Sprint 2 **S3** Sprint 3

EPIC-1 **Document Ingestion & Knowledge Base** 7 items

STORY	DT-1.1	As a Scrum Master, I can upload an SRS PDF/DOCX so the system ingests it automatically. — *File upload UI + backend parser*	S1
TASK	DT-1.2	Implement PDF/DOCX text extraction using PyMuPDF or python-docx.	S1
TASK	DT-1.3	Chunk extracted text into 400–600 token segments with overlap.	S1
TASK	DT-1.4	Generate vector embeddings using OpenAI text-embedding-3-small or HuggingFace sentence-transformers.	S1
TASK	DT-1.5	Store vectors + metadata in ChromaDB / Pinecone with project namespace isolation.	S1
STORY	DT-1.6	As a Scrum Master, I can see a list of uploaded documents per project with ingestion status.	S1
TASK	DT-1.7	Support re-ingestion when document is updated (version diff + re-embed changed chunks).	S2

EPIC-2 AI Story Generation (RAG + LLM) — 8 items

STORY	DT-2.1	As a Scrum Master, I can type a module name and get auto-generated Epics, Stories & Tasks. — *Core AI feature*	S1
TASK	DT-2.2	Build RAG pipeline: retrieve top-K relevant chunks from KB → inject into LLM prompt context.	S1
TASK	DT-2.3	Design structured LLM prompt template that outputs strict JSON: {epics[], stories[], tasks[], acceptance_criteria[]}.	S1
TASK	DT-2.4	Validate LLM JSON output schema — retry with corrective prompt if malformed.	S2
STORY	DT-2.5	As a Scrum Master, I can review, edit, delete, or re-generate any story before pushing to Jira.	S2
STORY	DT-2.6	As a Scrum Master, I can assign story points and priority to generated stories before approval.	S2
TASK	DT-2.7	Add budget/deadline context to LLM prompt so story estimates respect project constraints.	S2
TASK	DT-2.8	Store generated story history per project for audit trail and re-use.	S3

EPIC-3 Jira Integration — 6 items

TASK	DT-3.1	Implement Jira OAuth 2.0 / API token authentication. Store tokens securely per user.	S2
STORY	DT-3.2	As a Scrum Master, I can link my Jira workspace to DevTrack by entering project key & API token.	S2
TASK	DT-3.3	Use Jira REST API v3 to create Epic → Story → Sub-task hierarchy with correct issue type mapping.	S2

STORY	DT-3.4	As a Scrum Master, I can push approved stories to Jira in one click.	S2
TASK	DT-3.5	Implement bidirectional sync — if story is manually updated in Jira, reflect in DevTrack dashboard.	S3
TASK	DT-3.6	Handle Jira rate-limiting, retry logic, and error notifications.	S3

EPIC-4 GitHub Code Analysis Engine

8 items

TASK	DT-4.1	GitHub App / OAuth integration — get repo access with read permissions on code & webhooks.	S2
STORY	DT-4.2	As a Scrum Master, I can link a GitHub repository to a DevTrack project.	S2
TASK	DT-4.3	Set up GitHub webhook listener — trigger analysis pipeline on every push to main/develop branch.	S2
TASK	DT-4.4	Parse changed files using AST (Babel for JS/TS, tree-sitter for multi-language) — extract routes, components, functions.	S2
TASK	DT-4.5	Build LLM validator: given a story's acceptance criteria + extracted code evidence, determine DONE / PARTIAL / NOT STARTED.	S3
STORY	DT-4.6	As a developer, I can see which file and line number is linked to each completed story (traceability).	S3
TASK	DT-4.7	Handle multi-language support: JavaScript, TypeScript, Python, Java — configurable per project.	S3
TASK	DT-4.8	Cache analysis results — only re-analyse files changed in the latest commit (delta analysis).	S3

EPIC-5 Real-Time Progress Dashboard

7 items

STORY	DT-5.1	As a Scrum Master, I see a dashboard with Epic-level progress bars and overall % completion.	S2
STORY	DT-5.2	As a developer, I see which of my assigned stories are Done, Partial, or Not Started based on my commits.	S3
STORY	DT-5.3	As a project manager, I see a sprint burndown chart (planned vs actual velocity).	S3
TASK	DT-5.4	Implement WebSocket (Socket.io) so dashboard updates in real-time when GitHub push is analysed.	S3
STORY	DT-5.5	As a Scrum Master, I can export the current progress report as a PDF.	S3
TASK	DT-5.6	Risk alerts: flag stories with no commits for 3+ days, or epics behind schedule.	S3
TASK	DT-5.7	Module heat-map: visual grid showing completion status per module (red/yellow/green).	S3

EPIC-6	Auth, Projects & User Management	5 items
TASK	DT-6.1	JWT-based authentication with role support: Admin, Scrum Master, Developer.	S1
STORY	DT-6.2	As an Admin, I can create a project and invite team members by email.	S1
STORY	DT-6.3	As a user, I can switch between multiple projects from a single account.	S2
TASK	DT-6.4	Role-based access: only Scrum Master can push to Jira; developers see read-only dashboard.	S2
TASK	DT-6.5	Audit log: record every AI generation, Jira push, and code analysis event per project.	S3

⚙️ 5. Tech Stack Recommendation

🖥️ Frontend

React 18 · Vite · Tailwind CSS · Recharts (graphs) · Socket.io-client · Axios

🔧 Backend (API Server)

Node.js + Express · Socket.io · JWT Auth · GitHub REST API · Jira REST API v3

🧠 AI / ML Layer

Python + FastAPI · LangChain · OpenAI GPT-4o · HuggingFace Embeddings · ChromaDB / Pinecone · PyMuPDF

🗄️ Database & DevOps

MongoDB Atlas · Redis (caching) · Docker · GitHub Actions CI/CD · Render / Railway (hosting)

> The AI service (Python/LangChain) runs as a separate microservice. Node.js backend handles API, auth, webhooks and calls the AI service when needed. This is the correct separation of concerns.

🔍 6. Competitor Analysis — What Exists vs What You're Building

Tool	Auto-reads SRS?	Generates Stories?	Monitors GitHub?	Auto-marks Done?	Jira Integration?
Jira (Atlassian)	✗ No	✗ No	⚠ Basic	✗ No	✓ It IS Jira
Linear	✗ No	⚠ Partial AI	✓ Yes	✗ No	⚠ Limited
GitHub Projects	✗ No	✗ No	✓ Native	✗ No	⚠ Limited
Azure DevOps	✗ No	✗ No	✓ Yes	✗ No	✓ Yes
Notion AI	⚠ Manual	⚠ Basic	✗ No	✗ No	⚠ Limited

✨ **DevTrack (yours)**	**✓ Yes (RAG)**	**✓ Yes (GPT-4)**	**✓ Yes (webhooks)**	**✓ Yes (AI validator)**	**✓ Yes (2-way sync)**

> No single existing tool does all 5 of these together. That is your competitive advantage.

⚖️ 7. Copyright & Novelty Assessment

✅ What Makes DevTrack Genuinely Novel (Publishable / Copyrightable)

- The **end-to-end pipeline**: SRS → AI Stories → Jira → GitHub → Auto-completion — no existing tool connects all these steps
- The **acceptance criteria validator**: Using LLM to check if code satisfies a story's criteria is not in any commercial tool yet
- The **RAG + Jira generation** combination: retrieving from project docs to generate structured Jira items is a novel workflow
- The **traceability layer**: linking a specific file + line to a user story is a research-level contribution
- The system architecture as a whole is an **original creative work** — copyrightable under Indian copyright law

⚠️ Copyright Notes You Must Understand

- **Ideas cannot be copyrighted** — only the implementation (your code, architecture diagrams, documentation) can be
- Your **source code is automatically copyrighted** the moment it is written — no registration needed in India
- For stronger protection, register with **Copyright Office of India** (online at copyright.gov.in) — costs ₹500 per work
- If you want patent protection for the method/algorithm, file a **provisional patent** with Indian Patent Office before publishing the paper
- The published research paper itself will be protected by the **journal/conference copyright** — retain your moral rights
- OpenAI, GitHub, and Jira APIs have their own ToS — ensure commercial use is permitted if you plan to monetize

🚨 8. Honest Gaps & Risks

✅ What's Strong

- Problem is real and unsolved

⚠️ Challenges to Solve

❌ Real Risks

- Full pipeline is well-defined
- RAG approach is industry-standard
- You already have React + Node skills
- HMS project = perfect test case
- Strong team size for scope

- LLM accuracy varies — need fallback prompts
- GitHub AST parsing is non-trivial
- Jira API has strict rate limits
- OpenAI API costs money at scale
- Code-to-story mapping can be ambiguous
- Need to support multiple languages

- Scope is very large for final year project
- AI accuracy hard to guarantee (evaluation needed)
- Dependency on paid APIs (budget risk)
- If Jira changes API, integration breaks
- No offline/self-hosted LLM option defined yet

Mitigation Strategy

Risk	Mitigation
Scope too large	Build MVP with only EPIC 1+2+5 for submission. EPIC 3+4 as "future work"
OpenAI cost	Use free-tier Gemini API or local Ollama (Mistral 7B) as fallback
LLM accuracy	Define BLEU/F1 evaluation metric on a test SRS — show accuracy % in paper
GitHub AST complex	Start with simple keyword matching → upgrade to AST in v2

🗺️ 9. Sprint-wise Development Roadmap

Sprint 1
Weeks 1–3

Foundation + AI Core

→ Project setup (React + Node + Python microservice + MongoDB)

→ JWT Auth + User/Project management (DT-6.1, 6.2)

→ Document upload + PDF extraction + Chunking + Embedding (DT-1.1 to 1.5)

→ Basic LLM prompt → generate stories JSON (DT-2.1 to 2.3)

→ Simple review UI to display generated stories

Goal: Demo — upload SRS → see generated stories

Sprint 2
Weeks 4–6

Jira Integration + GitHub Monitoring

→ Jira OAuth + push stories to Jira (DT-3.1 to 3.4)

→ GitHub webhook setup + file change detection (DT-4.1 to 4.4)

→ Basic dashboard with Epic progress bars (DT-5.1)

→ Story review/edit/approve workflow (DT-2.5, 2.6)

→ Multi-project support (DT-6.3, 6.4)

Goal: Demo — stories auto-created in Jira, GitHub pushes tracked

Sprint 3
Weeks 7–9

AI Completion Validator + Full Dashboard

→ LLM code → story acceptance criteria matcher (DT-4.5)

→ Traceability (file + line → story link) (DT-4.6)

→ Real-time WebSocket dashboard updates (DT-5.4)

→ Burndown chart + risk alerts (DT-5.3, 5.6)

→ PDF export of progress report (DT-5.5)

→ Bidirectional Jira sync (DT-3.5)

Goal: Full end-to-end demo — SRS to completed story tracking

Sprint 4
Weeks 10–12

Testing, Evaluation & Paper

→ Test on real project (use HMS as test case!)

→ Measure AI accuracy (precision/recall on story generation)

→ Performance testing (concurrent users, webhook load)

→ Write research paper: problem, methodology, results, comparison

→ Final presentation + demo video

Goal: Submission-ready product + published/submitted paper

🏆 10. Final Verdict

Is the idea great? — Yes. It solves a real, daily pain point for every software team. The combination of RAG + Jira + GitHub code analysis in one automated pipeline does not exist as a complete product today.

Is the copyright valid? — Yes. Your implementation, architecture, and documentation are original creative works protected under the Copyright Act of India 1957. The research paper adds academic novelty. Consider filing a provisional patent for the acceptance-criteria-to-code validation method.

Main risk? — Scope. Prioritize: get the SRS → Story generation → Dashboard working first. Jira + GitHub integration is impressive but secondary. A working core beats a half-finished full system.

Your MediCore HMS project is the perfect real-world test case to validate DevTrack. Run DevTrack *on* the HMS project — use the HMS SRS as the input document, and show how DevTrack would have generated the same Jira plan that was created manually. That demo story alone makes a strong research paper.